The boss wants you and Bill to review the source of a small application called Bookworm. The company developed Bookworm to manage its large internal library of books. The program has slowly grown out of control as many different developers have added their pet features to the mix, from text previews to magazine management and the tracking of borrowed books. As a result, the Bookworm source code is a bit of a mess. You and Bill have been selected to go through the code and clean it up. The boss called it "just an easy refactoring job."

You and Bill have been browsing through the Bookworm source code for a few minutes when you spot your first refactoring opportunity. Bookworm has a function that formats book titles for printing on old-fashioned tape labels. It strips all punctuation and special characters out of a string, leaving only alphanumeric characters and spaces:

object_model/alphanumeric.rb

```
def to_alphanumeric(s)
  s.gsub(/[^\w\s]/, '')
end
```

This method also comes with its own unit test (remember to gem install test-unit before you try to run it on Ruby 2.2 and later):

```
require 'test/unit'

class ToAlphanumericTest < Test::Unit::TestCase
  def test_strip_non_alphanumeric_characters
    assert_equal '3 the Magic Number', to_alphanumeric('#3, the *Magic, Number*?')
  end
end
```

"This to_alphanumeric method is not very object oriented, is it?" Bill says. "This is generic functionality that makes sense for all strings. It'd be better if we could ask a String to convert itself, rather than pass it through an external method."

Even though you're the new guy on the block, you can't help but interrupt. "But this is just a regular String. To add methods to it, we'd have to write a whole new AlphanumericString class. I'm not sure it would be worth it."

"I think I have a simpler solution to this problem," Bill replies. He opens the String class and plants the to_alphanumeric method there:

```
class String
  def to_alphanumeric
    gsub(/[^\w\s]/, '')
  end
end
```